

Documentacion práctica 3

Proyecto: Sistema de Alarma para Incubadora de Bebé (ESP32 + MicroPython)

Integrantes:

- Felipe Villa Jaramillo
- Sara Pineda
- Juan Pablo Cardona Bedoya

Este documento describe la implementación de un sistema de monitoreo de temperatura y voltaje para una incubadora neonatal, utilizando un ESP32 y MicroPython. El proyecto cumple con las rúbricas especificadas en la guía del laboratorio.

Objetivos

- Leer temperatura desde **tres fuentes redundantes** (LM35, MAX6675, BMP280*).
- Medir voltaje de entrada usando un **potenciómetro**.
- Activar LEDs de alerta para diferentes niveles de temperatura y voltaje.
- Transmitir datos por **UART** al PC.
- Permitir seleccionar el sensor activo vía archivo de configuración.

Nota: Se omite BMP280 por instrucción explícita.

Componentes utilizados

- ESP32
 - Sensor de temperatura **LM35** (analógico)
 - Sensor de temperatura **MAX6675** + termocupla (SPI)
 - Potenciómetro (para simular voltaje)
 - 4 LEDs (verde, amarillo, rojo, azul voltaje)
 - Módulo USB-UART
 - Resistencias para LEDs (~220Ω)
 - Cables y protoboard
-

Asignación de pines

Elemento	Pin ESP32	Descripción
LM35	GPIO34	Entrada ADC
Potenciómetro (Voltaje)	GPIO35	Entrada ADC
MAX6675 - SCK	GPIO14	Reloj SPI
MAX6675 - CS	GPIO15	Chip Select
MAX6675 - SO (MISO)	GPIO12	Salida de datos SPI
LED Verde (Normal)	GPIO21	Salida digital
LED Amarillo (Warning)	GPIO22	Salida digital
LED Rojo (Emergencia)	GPIO23	Salida digital
LED Azul (Voltaje bajo)	GPIO2	Salida digital
UART TX (ESP32 → PC)	GPIO17	Transmisión UART
UART RX (PC → ESP32)	GPIO16	Recepción UART

Conexiones físicas

LM35

- VCC → 3.3V
- GND → GND
- VOUT → GPIO34
- Capacitor cerámico de 100nF entre VOUT y GND

Potenciómetro

- Extremo 1 → 3.3V
- Extremo 2 → GND
- Medio (wiper) → GPIO35

MAX6675

- VCC → 3.3V
- GND → GND
- SCK → GPIO14
- CS → GPIO15
- SO → GPIO12

LEDs

- LED Verde (Normal) -> GPIO21

- LED Amarillo (Warning) -> GPIO22
- LED Rojo (Emergencia) -> GPIO23
- LED Rojo (Voltaje bajo) -> GPIO2

UART a USB

- GPIO17 (TX) → RX del convertidor
- GPIO16 (RX) → TX del convertidor
- GND → GND del convertidor

Descripción de módulos del sistema

El código está desarrollado en un solo archivo y separado por diferentes clases para la correcta utilización del mismo, desglosaremos cada uno de estos.

Esta clase maneja todo lo relacionado con el sensor de la termocupla, básicamente se encarga de inicializar los pines **miso**, **sck**, **mosi** **cs** por donde se comunicará el sensor con el **ESP32**.

```
import ujson
import time
import _thread
from machine import ADC, Pin, UART, SoftSPI

class Thermocouple:
    def __init__(self, miso_pin=12, sck_pin=14, mosi_pin=13, chip_select_pin=15):
        self.bus = SoftSPI(sck=Pin(sck_pin), mosi=Pin(mosi_pin),
miso=Pin(miso_pin))
        self.cs = Pin(chip_select_pin, Pin.OUT)
        self.cs.value(1)
```

Este es el método encargado de tomar las medidas para el sensor, "despellizca" y así escucha la medida que se guarda en **raw** para así después hacerle un corrimiento y obtener el valor real que es el que será retornado.

```
def read_temp(self):
    self.cs.value(0)
    time.sleep_us(10)
    raw = self.bus.read(2)
    self.cs.value(1)

    value = (raw[0] << 8 | raw[1]) >> 3
    return value * 0.25 # °C
```

A continuación ya pasamos con la clase genérica para ADC que es donde manejamos tanto el **potenciometro** como el sensor **LM35**. Decidimos manejar ambos dentro de esta misma clase ya que

así estamos evitando repetir código porque estos sensores tienen características comunes y que son ambos sensores analógicos.

Este inicializa una **escala** y una **resolución**, la primera sirve para tener el contexto que en este caso el potenciómetro oscilará entre 0 y 3.3 V y la segunda corresponde a la resolución del ADC del ESP32, que es de 12 bits lo que esto significa que el rango de valores digitales posibles va de 0 a 4095 (es decir, $2^{12} - 1$). Se utilizan juntos para convertir el valor crudo leído del ADC (de 0 a 4095) en un voltaje real (de 0 a 3.3 V) usando la fórmula: $\text{voltage} = (\text{raw} / \text{resolution}) * \text{scale}$

```
class GenericADC:
    def __init__(self, pin, scale=3.3, resolution=4095):
        self.adc = ADC(Pin(pin))
        self.adc.atten(ADC.ATTN_11DB)
        self.scale = scale
        self.resolution = resolution
        self.value = 0
```

Seguidamente contamos con los métodos de lectura, en primer lugar encontramos el método de lectura del potenciómetro que básicamente toma la lectura y se convierte por medio de la fórmula anteriormente mencionada para que el voltaje se obtenga correctamente.

```
def read(self):
    raw = self.adc.read()
    voltage = (raw / self.resolution) * self.scale
    self.value = voltage
    return voltage
```

En segundo lugar tenemos la lectura del LM35, que básicamente lo que hace es generar un voltaje proporcional a la temperatura ambiente entonces, cuando se mide la señal del LM35 con el ADC, se obtiene un voltaje como por ejemplo 0.28 V, y se convierte a temperatura con la fórmula: $\text{voltaje} * 100$ y se multiplica por 100 ya que por cada grado Celsius que aumenta la temperatura, el voltaje de salida del LM35 aumenta 0.01 voltios (o 10 milivoltios).

```
def read_temp_lm35(self):
    voltage = self.read()
    return voltage * 100 # LM35: 10mV/°C
```

Ahora pasamos con la clase que se encarga de controlar los LEDS

```
class LEDController:
    def __init__(self, config_path="config.json"):
        self.load_config(config_path)
        self.green = Pin(21, Pin.OUT)
        self.yellow = Pin(22, Pin.OUT)
```

```
self.red_temp = Pin(23, Pin.OUT)
self.red_voltage = Pin(2, Pin.OUT)
```

Después de tener su inicialización podemos cargar su configuración, la cual la cargamos desde el archivo `config.json` En este podemos definir en qué medidas se encenderán los leds por temperaturas muy altas o riesgosas; a su vez definimos el voltaje mínimo que puede tener antes que se encienda el led que indicará un bajo voltaje y por último el sensor que se utilizará por defecto en las medidas.

```
{
  "warning_level": 34.0,
  "emergency_level": 35.0,
  "low_voltage_threshold": 1.0,
  "sensor_source": "Thermocouple"
}
```

De aquí en el siguiente método cargamos la configuración detectando cada uno de los valores que extrajimos del `config.json` y asignandoselos a la clase

```
def load_config(self, path):

    with open(path) as f:
        config = ujson.load(f)
        self.warning_level = config["warning_level"]
        self.emergency_level = config["emergency_level"]
        self.low_voltage_threshold = config["low_voltage_threshold"]
        self.sensor_source = config["sensor_source"]
```

Una vez teniendo la configuración ya podemos tener el método de actualización de los leds, en donde controlamos cada uno de los eventos donde se tiene que encender:

- En primer lugar si la temperatura es menor que el nivel de precaución quiere decir que tenemos que encender únicamente el led de color **verde**

```
if temp < self.warning_level:
    self.green.value(1)
    self.yellow.value(0)
    self.red_temp.value(0)
```

- En segundo lugar, si la temperatura es mayor que el nivel de emergencia pero menor al nivel de precaución, esto quiere decir que debemos encender el led amarillo

```
elif self.warning_level <= temp < self.emergency_level:
    self.green.value(0)
    self.yellow.value(1)
    self.red_temp.value(0)
```

- En tercer lugar, si ninguno de los dos anteriores casos se cumple, quiere decir que debemos de encender el led **rojo** para indicar que estamos en temperaturas de emergencia.

```
else:
    self.green.value(0)
    self.yellow.value(0)
    self.red_temp.value(1)
```

- En cuarto lugar, para encender hacemos una validación de un condicional para encender el led de precaución de bajo voltaje, el cual valida que si el nivel del voltaje por debajo del permitido entonces enciende el led y si está por encima lo apaga

```
if voltage < self.low_voltage_threshold:
    self.blue_low_warning_led.value(1)
else:
    self.blue_low_warning_led.value(0)
```

Ahora vamos a pasar a la clase donde manejamos el protocolo de comunicación UART, donde inicialmente asignamos el número de los pines que tenemos para comunicarnos con el **ESP32**

```
class UARTLogger:
    def __init__(self, tx=17, rx=16, baudrate=9600):
        self.uart = UART(1, tx=tx, rx=rx, baudrate=baudrate)
```

Aquí vamos a definir el mensaje que se mostrará en el **PuTTY** donde lo que hacemos es establecer el momento en el que se tomó la medida, mostrar las temperaturas de los dos sensores pero diciendo cual es el sensor que se encuentra en uso en este momento. Finalmente hacemos el `uart.write()` para que se envíe por UART hacia PuTTY

```
def log(self, temp_lm35, voltage, temp_thermocouple):
    msg = ujson.dumps({
        "timestamp": time.time(),
        "LM35_temp": temp_lm35,
        "Thermocouple_temp": temp_thermocouple,
        "Voltage": voltage,
        "sensor_in_use": "LM35" if temp_lm35 else "Thermocouple"
    })
    self.uart.write("\n" + msg)
```

Ahora ya pasamos con la función que toma las medidas en tanto del sensor **LM35** y el **potenciometro**

```
def adc_worker(adc_lm35, adc_voltage):
    while True:
        adc_lm35.read()
        adc_voltage.read()
        time.sleep(0.1)
```

Ya en cuanto a la función principal del programa lo que hacemos es inicializar los sensores que tenemos

```
def main():  
    # Inicialización de sensores  
    lm35 = GenericADC(pin=34)  
    voltage = GenericADC(pin=35)  
    thermocouple = Thermocouple()  
    led_controller = LEDController()  
    uart_logger = UARTLogger()
```

Después inicializamos el nuevo hilo para tomar las medidas de ADC en **paralelo**

```
_thread.start_new_thread adc_worker, (lm35, voltage))
```

Ahora inicializamos el bucle para que nuestro programa se ejecute indefinidamente y con la lectura de los sensores. Esta lectura viene del **ADC**, que además ya está siendo actualizado constantemente por el hilo paralelo además de tomar el valor actual del voltaje, que fue actualizado también por el hilo paralelo y después toma la temperatura actual de la **termocupla** en ese momento (no es en paralelo porque no se especifica en las rúbricas)

```
while True:  
    temp_lm35 = lm35.read_temp_lm35()  
    voltage_val = voltage.value  
    temp_tc = thermocouple.read_temp()
```

Ahora determinamos cual es el sensor que se está usando para encender los LEDs, si se tiene que el nombre es **LM35** entonces se usa este y si tiene otro, entonces se utiliza la termocupla.

```
if led_controller.sensor_source == "LM35":  
    temp_to_use = temp_lm35  
else:  
    temp_to_use = temp_tc
```

Aquí es donde actualizamos constantemente los LEDs y enviamos los datos por UART para que se puedan obtener la información y los LEDs se enciendan o se apaguen según la situación específica.

```
led_controller.update_leds(temp_to_use, voltage_val)  
uart_logger.log(temp_lm35, voltage_val, temp_tc)
```

Por último tenemos un print por consola **únicamente por pruebas** en donde mostramos el valor de todos los sensores.

```
print(f"LM35: {temp_lm35:.2f}°C | Termopar: {temp_tc:.2f}°C | Voltaje:  
{voltage_val:.2f}V")
```

```
time.sleep(1)
```

Por último detectamos si el programa donde se ejecuta la aplicación es el principal y si es así entonces inicializa la aplicación

```
if __name__ == "__main__":  
    main()
```